Diffusion Models: From Adding Noise to Generating Images

Vong Chi Van Le Dat Phuc An
Faculty of Information Technology, University of Science, VNU-HCM
{24125049, 24125052}@student.hcmus.edu.vn

Abstract

Diffusion models have emerged as a dominant paradigm in generative artificial intelligence, achieving unprecedented visual fidelity and robust likelihood-based training without adversarial instability. This report investigates the fundamental guiding question: *how can learning to remove noise enable a neural network to generate novel images, and why does diffusion within a learned latent space drastically reduce computational requirements?* Grounded in two foundational papers, Denoising Diffusion Probabilistic Models (Ho et al., 2020) and Latent Diffusion Models (Rombach et al., 2022), we examine the mathematical principles of forward Gaussian corruption, reverse denoising, and perceptual latent compression. Through theoretical derivations, a step-by-step numerical worked example, and empirical benchmark experiments, we provide an integrated analysis of diffusion dynamics, high-resolution scaling trade-offs, and practical deployment limitations.

Keywords: Generative AI, Denoising Diffusion Probabilistic Models, Latent Diffusion Models, Variational Autoencoders, Cross-Attention, Generative Learning Trilemma.

1 Introduction

Synthesizing high-dimensional continuous signals such as natural images is a central challenge in artificial intelligence. A natural image tensor is represented as $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, where H, W, and C denote the height, width, and color channel dimensions, respectively. Unlike supervised discriminative learning, which only seeks decision boundaries separating predefined class labels, a generative model must capture the true underlying probability distribution $p_{\text{data}}(\mathbf{x})$ of a complex, low-dimensional data manifold embedded in high-dimensional ambient space. Once learned, the model enables the synthesis of novel, realistic samples $\mathbf{x}_{\text{new}} \sim p_\theta(\mathbf{x})$.

Deep generative architectures have historically struggled to simultaneously achieve high sample sharpness, comprehensive mode diversity, and fast sampling speed. While early generative paradigms made substantial progress, each faced inherent trade-offs between sample quality, training stability, and sampling efficiency.

Tasks and Report Organization. To systematically address the guiding question in accordance with the seminar requirements, this report is structured around six concrete tasks:

- *Task 1: Foundational Paradigms and the Generative Trilemma.* We contrast discriminative classifiers with generative modeling and compare autoregressive models, variational autoencoders, generative adversarial networks, and diffusion models across the generative learning trilemma both qualitatively and quantitatively (Sections 2.1, 2.2, and 4.2).
- *Task 2: DDPM Mathematical Framework.* We formulate the variance-preserving forward process, variance schedule selection, closed-form multi-step transitions, the learned reverse Markov chain, and the simplified noise-prediction training objective (Section 3.1).
- *Task 3: Symbol Definitions and 1D Worked Example.* We specify complete symbol definitions, physical units, and modeling assumptions, accompanied by a step-by-step one-dimensional numerical worked example illustrating stochastic signal contraction and reverse denoising (Section 3.2).
- *Task 4: Multi-Level Forward Corruption and Reverse Image Generation.* We demonstrate forward noise corruption across multiple timesteps and iterative reverse image generation trajectories on empirical benchmarks (Section 4.3).
- *Task 5: Latent Diffusion Architecture.* We analyze and empirically validate the two-stage inductive bias of Latent Diffusion Models, examining perceptual autoencoder compression, latent manifold smoothness, and cross-attention conditioning with classifier-free guidance (Sections 3.3 and 4.4).

- *Task 6: Pixel vs. Latent Comparative Evaluation and Practical Limitations.* We provide an evidence-supported comparison between pixel-space and latent-space diffusion across explicit computational and quality metrics, followed by a critical evaluation of real-world limitations including sequential sampling latency, prompt attribute binding, copyright memorization, and societal bias (Sections 4.5 and 5).

2 Background and Paper Review

2.1 Distinguishing Discriminative vs. Generative Models

Supervised machine learning and probabilistic modeling are fundamentally divided into two complementary paradigms:

1. *Discriminative Models:* Discriminator models are mainly used for classification and regression tasks. A discriminative model directly learns to predict the conditional probability distribution $P(Y \mid \mathbf{x})$, where Y is the class label and $\mathbf{x}$ represents the input features. It learns decision boundaries separating classes without modeling the distribution of the input features $\mathbf{x}$. Consequently, a discriminative model cannot generate novel data samples and often produces spuriously overconfident predictions on empty out-of-distribution regions.

2. *Generative Models:* Generative models are primarily used to generate novel data, though they can also be used for classification. A generative model learns the underlying data distribution $p(\mathbf{x})$, or the joint distribution $p(\mathbf{x}, Y)$ when conditioned on class labels Y. Its primary function is synthesizing new, realistic samples $\mathbf{x}_{\text{new}} \sim p(\mathbf{x})$ that resemble the training data distribution, while assigning negligible density to regions devoid of data.

Figure 1 illustrates this fundamental distinction on a four-class two-dimensional manifold.

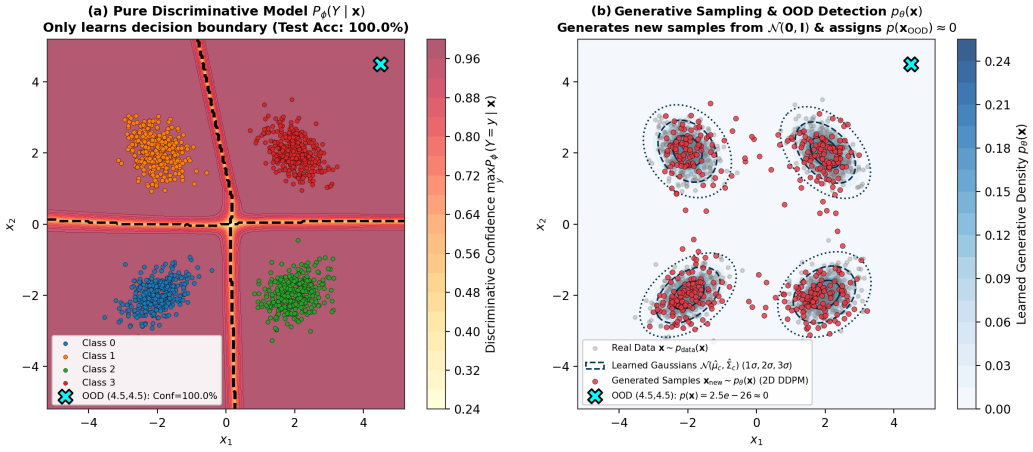


Figure 1. (a) A discriminative classifier learns quadrant decision boundaries to predict $P(Y \mid \mathbf{x})$ for classification, but exhibits unwarranted 100.0% overconfidence at an empty out-of-distribution coordinate $\mathbf{x}_{\text{OOD}} = (4.5, 4.5)$ and lacks generative ability; (b) A generative model explicitly learns the data density $p_\theta(\mathbf{x})$, detects out-of-distribution queries by assigning negligible density ($p(\mathbf{x}_{\text{OOD}}) \approx 0$), and synthesizes novel realistic samples $\mathbf{x}_{\text{new}} \sim p_\theta(\mathbf{x})$ along all four data modes.

2.2 Qualitative comparison of GANs, autoregressive models, and diffusion models

To contextualize diffusion models within the generative literature, we compare autoregressive models, variational autoencoders (VAEs), generative adversarial networks (GANs), and diffusion models across the Generative Learning Trilemma [16], which posits that a generative model ideally satisfies three core properties: (1) *High Sample Quality* (generating sharp, photorealistic samples), (2) *Fast Sampling* (synthesizing samples with minimal forward passes), and (3) *Full Mode Coverage* (faithfully covering all modes of the data distribution without mode collapse).

Historically, generative architectures achieved at most two of these three objectives, a theoretical trade-off that we also verify directly in our empirical experiments (Sections 4.2–4.5):

1. *Autoregressive Models (PixelRNN, PixelCNN) [10, 11]:* Factorize the joint distribution pixel-by-pixel via the probability chain rule $p_\theta(\mathbf{x}) = \prod_{i=1}^{n} p_\theta(x^{(i)} \mid x^{(<i)})$. They maximize exact log-likelihood and avoid mode collapse, achieving sharp local contrast and full diversity. However, sequential pixel-by-pixel generation incurs $\mathcal{O}(n)$ complexity and extremely slow sampling, as confirmed in our MNIST benchmark (Section 4.2, Figure 5 and Table 3), where PixelCNN requires 784 sequential forward passes ($4{,}294.7$ ms per 64 images).

2. *Variational Autoencoders (VAEs) [4]:* Introduce a continuous latent variable $\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ with an encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ and decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$ trained via the evidence lower bound (ELBO). VAEs generate images

in a single decoder pass and cover all modes, but pixel-level reconstruction losses average across plausible modes and produce visibly blurry samples, reflected in our benchmark by the fastest sampling latency (6.5 ms per 64 images) but the lowest edge sharpness (LapVar $= 0.0182$, FID $= 82.27$ in Table 3).

3. *Generative Adversarial Networks (GANs) [5, 6]:* Frame synthesis as a minimax game $\min_G \max_D$ between a Generator $G_\theta(\mathbf{z})$ and Discriminator $D_\phi(\mathbf{x})$. GANs produce razor-sharp images in a single feed-forward step (11.4 ms per 64 images and LapVar $= 0.0541$ in Table 3), but the lack of an explicit density objective causes training instability and mode dropping (FID $= 44.10$) [8].

4. *Denoising Diffusion Probabilistic Models (DDPM) [3, 1]:* Invert a parameterized T-step Markov chain using a U-Net $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ trained via unweighted mean-squared error L_{simple}. Diffusion models achieve state-of-the-art visual fidelity and full mode coverage, attaining the highest sharpness (LapVar $= 0.0601$), highest classifier confidence (97.4%), and lowest FID (16.85) in our MNIST benchmark (Table 3), with sequential sampling latency (3,926.8 ms for $T = 300$ steps) as their primary bottleneck.

5. *Latent Diffusion Models (LDM) [2]:* Resolve the computational bottleneck of DDPM by compressing $\mathbf{x}_0 \to \mathbf{z}_0$ via a frozen perceptual spatial VAE and executing diffusion on the low-dimensional latent tensor $\mathbf{z}_t$. Our experiments in Sections 4.4 and 4.5 (Figures 7–10) demonstrate that this spatial latent bottleneck preserves perceptual fidelity (PSNR $= 33.74$ dB, FID $= 0.18$) while accelerating U-Net training by $7.61\times$ per epoch and cutting GPU activation memory by $13.81\times$.

Table 1 summarizes the qualitative properties of each generative family across the trilemma, citing primary literature sources for every claim, while Section 4.2 (Figure 5 and Table 3) and Sections 4.4–4.5 (Figures 7–10 and Table 4) present our corresponding qualitative and quantitative experimental validations.

Table 1. Qualitative comparison of major generative model families across the Generative Learning Trilemma [16], citing primary literature sources for each theoretical property and empirical claim (Task 1).

Model Family	Density $p_\theta(\mathbf{x})$	Sample Quality	Mode Coverage	Sampling Speed
Autoregressive [10, 11] (PixelRNN, PixelCNN)	Explicit, Exact (Exact MLE) [10, 11]	Sharp local contrast [10, 16]	Full (No mode collapse; stable MLE) [10, 1]	Extremely Slow ($\mathcal{O}(n)$ sequential pixel evaluations) [10, 13, 16]
VAE [4] (Variational Autoencoder)	Explicit, Approx. (ELBO Bound) [4]	Blurry (Single-step L_2/BCE mode averaging) [4, 9, 16]	Full, smooth latent manifold [4, 16]	Very Fast (1 Decoder forward pass) [4, 16]
GAN [5, 6] (DCGAN, StyleGAN)	Implicit (Likelihood-free) [5]	Razor Sharp (Mode-seeking via adversarial loss) [5, 7, 16]	Poor (Prone to Mode Collapse and minimax instability) [5, 8, 16]	Very Fast (1 Generator forward pass) [5, 16]
Pixel Diffusion [3, 1] (DDPM)	Explicit, Approx. (T-step ELBO) [3, 1]	SOTA Sharpness (FID 3.17 on CIFAR-10) [1, 14]	Full (Stable regression; beats GAN mode coverage) [1, 14, 16]	Slow ($T \approx 1000$ sequential full-res U-Net passes) [1, 13, 16]
Latent Diffusion [2] (LDM / Stable Diffusion)	Two-Stage ELBO (VAE + Diffusion) [2]	SOTA Photorealism at high resolution (FID 5.11 on CelebA-HQ) [2]	Full and flexible multimodal text conditioning [2]	Fast and Practical (Operates on $48\times$ smaller $\mathbf{z}_t$; 20 to 50 DDIM steps) [2, 13]

2.3 Foundational Literature Review

This report analyzes two foundational milestone papers:

- *Required Paper: Ho, Jain and Abbeel (NeurIPS 2020), "Denoising Diffusion Probabilistic Models" (DDPM) [1]:* Demonstrated that diffusion models generate image quality rivaling GANs while retaining likelihood-based mode coverage. The paper made three key contributions: (1) establishing the equivalence between denoising score matching and Markov chain reverse step estimation; (2) reparameterizing the reverse transition mean to predict the injected Gaussian noise vector $\boldsymbol{\epsilon}$; (3) introducing the unweighted simplified training objective $L_{\text{simple}}(\theta)$, which suppresses small-step noise artifacts and prioritizes perceptual visual features.

- *Recommended Paper: Rombach et al. (CVPR 2022), "High-Resolution Image Synthesis With Latent Diffusion Models" (LDM) [2]:* Identified that training pixel-space diffusion models on high-resolution images allocates excessive compute to imperceptible high-frequency details. Rombach et al. introduced a two-stage paradigm: (1) a first stage Variational Autoencoder (VAE) trained with perceptual and adversarial losses that compresses high-dimensional images into an efficient spatial latent space; (2) a second stage diffusion backbone operating exclusively on latent representations, incorporating cross-attention layers to enable flexible conditioning on text prompts, bounding boxes, or segmentation maps.

3 Method and Step-by-Step Worked Example

3.1 Forward Process, Variance Schedule, Reverse Denoising, and Sampling (Task 2)

Forward Noising Process. Let clean data follow an unknown distribution $\mathbf{x}_0 \sim q(\mathbf{x}_0)$. The forward diffusion process is a non-trainable Markov chain governed by a fixed variance schedule $\beta_1, \beta_2, \ldots, \beta_T \in (0, 1)$, where the joint distribution and individual Gaussian transitions are defined as:

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1}), \quad q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\big(\mathbf{x}_t;\ \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\ \beta_t\mathbf{I}\big) \tag{1}$$

Variance-Preserving Property. Equation (1) implements a Variance-Preserving (VP) stochastic process. By scaling the incoming signal by $\sqrt{1-\beta_t}$, total variance is preserved: if the previous state has unit variance $\mathrm{Var}(\mathbf{x}_{t-1}) = \mathbf{I}$, then $\mathrm{Var}(\mathbf{x}_t) = (\sqrt{1-\beta_t})^2\mathrm{Var}(\mathbf{x}_{t-1}) + \beta_t\mathbf{I} = (1-\beta_t)\mathbf{I} + \beta_t\mathbf{I} = \mathbf{I}$. Consequently, the signal amplitude never explodes, and the final state converges smoothly to standard white Gaussian noise $\mathbf{x}_T \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Closed-Form Multi-Step Transition. A key property of Gaussian Markov chains is that intermediate steps can be bypassed in closed form. Defining $\alpha_t = 1 - \beta_t$ and cumulative signal retention $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, any intermediate noisy state $\mathbf{x}_t$ can be sampled directly from $\mathbf{x}_0$ using standard Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_t;\ \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\ (1-\bar{\alpha}_t)\mathbf{I}\big) \iff \mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon} \tag{2}$$

Equation (2) enables efficient training by sampling random timesteps $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$ in $\mathcal{O}(1)$ time without simulating steps 1 to $t-1$.

Variance Schedule Selection. The schedule $\beta_1, \ldots, \beta_T$ controls the rate of information destruction:

- *Small β_t and Large T:* Small β_t ensures that the true reverse transition $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ remains nearly Gaussian, while sufficiently large T ensures $\bar{\alpha}_T \approx 0$ so that $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
- *Linear Schedule (Ho et al. [1]):* Linearly interpolates from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$ across $T = 1000$.
- *Cosine Schedule (Nichol & Dhariwal [15]):* Sets $\bar{\alpha}_t = f(t)/f(0)$ with $f(t) = \cos^2\big(\frac{t/T+s}{1+s} \cdot \frac{\pi}{2}\big)$ to prevent abrupt signal destruction in early timesteps.
- *Latent Diffusion Schedule (Rombach et al. [2]):* Uses a scaled-linear schedule ($\beta_1 = 0.00085 \to \beta_T = 0.012$) tailored to normalized latent spaces.

Reverse Denoising Process and Posterior Tractability. The marginal reverse transition $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ is intractable because it requires integrating over the unknown data distribution $q(\mathbf{x}_0)$. However, conditioning on the clean image $\mathbf{x}_0$ via Bayes' rule renders the forward posterior Gaussian and analytically tractable:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0)\, q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}{q(\mathbf{x}_t \mid \mathbf{x}_0)} = \mathcal{N}\big(\mathbf{x}_{t-1};\ \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0),\ \tilde{\beta}_t\mathbf{I}\big) \tag{3}$$

Matching quadratic Gaussian exponents yields the closed-form posterior mean and variance:

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\mathbf{x}_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,\mathbf{x}_t, \quad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t \tag{4}$$

Since $\mathbf{x}_0$ is unknown during inference, we train a parameterized transition $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\big(\mathbf{x}_{t-1};\ \boldsymbol{\mu}_\theta(\mathbf{x}_t, t),\ \sigma_t^2\mathbf{I}\big)$ with $\sigma_t^2 = \tilde{\beta}_t$ (or β_t). Rearranging Equation (2) as $\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}\big(\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}\big)$ and substituting into Equation (4) reparameterizes $\boldsymbol{\mu}_\theta$ in terms of a U-Net noise predictor $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$, yielding the reverse sampling update step from $\mathbf{x}_t$ to $\mathbf{x}_{t-1}$:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) \implies \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t\mathbf{z} \tag{5}$$

where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for $t > 1$, and $\mathbf{z} = \mathbf{0}$ at the final step $t = 1$.

Score-Matching Equivalence and Simplified Objective L_{simple}. By Tweedie's Formula and denoising score matching [12, 1], the predicted noise estimates the Stein score of the perturbed density, $\mathbf{s}_\theta(\mathbf{x}_t, t) = -\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)/\sqrt{1-\bar{\alpha}_t} \approx \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)$, so subtracting predicted noise performs a step toward higher data density. Optimizing the variational lower bound L_{VLB} over the KL divergence terms $L_{t-1} = D_{\text{KL}}\big(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)\big)$ reduces to a weighted MSE on $\boldsymbol{\epsilon}$. Ho et al. [1] showed that dropping the timestep weighting prefactor improves sample quality by emphasizing perceptually salient larger-t denoising tasks, yielding the simplified objective:

$$L_{\text{simple}}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},\ t\big)\right\|^2\right] \tag{6}$$

where $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$, $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, and $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Figure 2 illustrates the resulting bidirectional Markov chain.

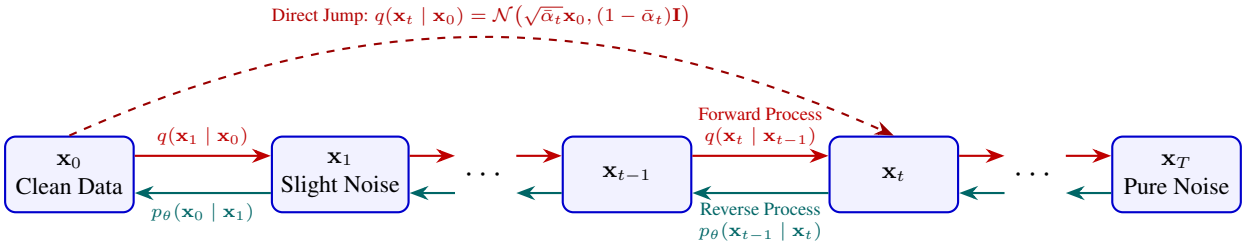


Figure 2. The DDPM Markov chain architecture: forward noising chain $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ (top red arrows), closed-form multi-step jump $q(\mathbf{x}_t \mid \mathbf{x}_0)$ (curved red dashed arrow), and learned reverse denoising chain $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ parameterized by U-Net $\boldsymbol{\epsilon}_\theta$ (bottom teal arrows) [1].

Training and Sampling Algorithms. Figure 3 presents the complete DDPM training and sampling procedures [1]. In *Algorithm 1 (Training)*, each optimization step samples a clean image $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, a uniform timestep $t \sim \text{Uniform}(\{1, \dots, T\})$, and standard Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, constructs the noisy observation $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$ directly in $\mathcal{O}(1)$ time via Equation (2), and updates the U-Net parameters θ by taking a gradient descent step on the unweighted MSE loss $\big\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\big\|^2$. In *Algorithm 2 (Sampling)*, generation starts from pure Gaussian noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and iteratively applies the learned reverse transition (Equation (5)) from $t = T$ down to 1—subtracting the predicted noise component and adding Langevin stochasticity $\sigma_t \mathbf{z}$ for $t > 1$ (with $\mathbf{z} = \mathbf{0}$ at the final step $t = 1$)—to produce a clean sample $\mathbf{x}_0$.

Algorithm 1 Training

repeat
  $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
  $t \sim \text{Uniform}(\{1, \dots, T\})$
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  Take gradient descent step on
  $\nabla_\theta \big\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon},\, t\big)\big\|^2$
until converged

Algorithm 2 Sampling

$\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
for $t = T, \dots, 1$ **do**
  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ **if** $t > 1$ **else** $\mathbf{z} = \mathbf{0}$
  $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\Big(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\Big) + \sigma_t \mathbf{z}$
end for
return $\mathbf{x}_0$

Figure 3. DDPM Training (Algorithm 1) and Sampling (Algorithm 2) matching Ho et al. [1].

3.2 Formal Equation Specifications and 1D Worked Numerical Example (Section III & Task 3)

Symbol Definitions and Modeling Assumptions. Table 2 provides complete mathematical definitions, units, value ranges, and explicit modeling assumptions for every symbol in the DDPM framework.

Step-by-Step One-Dimensional Worked Example. To verify the equations concretely, consider a one-dimensional continuous coordinate ($D = 1$) with initial clean signal $x_0 = 0.80$ at timestep $t = 1$, forward variance parameter $\beta_1 = 0.19$ (so $\alpha_1 = \bar{\alpha}_1 = 1 - \beta_1 = 0.81$ and $\sqrt{\alpha_1} = 0.90$), and sampled standard Gaussian noise $\epsilon = -0.50 \sim \mathcal{N}(0, 1)$:

- *Step 1: Forward Distribution $q(x_1 \mid x_0 = 0.80)$:* From Equation (1), $q(x_1 \mid x_0) = \mathcal{N}\big(x_1; \sqrt{1-\beta_1}\,x_0, \beta_1\big)$. Substituting the parameters yields mean $\mu_{x_1} = \sqrt{1-\beta_1}\,x_0 = 0.90 \times 0.80 = 0.7200$ and standard deviation $\sigma_{x_1} = \sqrt{\beta_1} = \sqrt{0.19} \approx 0.4359$, so $q(x_1 \mid x_0 = 0.80) = \mathcal{N}(x_1;\, 0.7200,\, 0.19)$.
- *Step 2: Realized Noisy Pixel x_1 for $\epsilon = -0.50$:* Applying the reparameterization formula in Equation (2):

$$x_1 = \sqrt{\bar{\alpha}_1}\,x_0 + \sqrt{1-\bar{\alpha}_1}\,\epsilon = 0.90(0.80) + 0.4359(-0.50) = 0.5021 \tag{7}$$

- *Step 3: Physical Interpretation of Stochastic Expectation $\mathbb{E}[x_1 \mid x_0] = 0.7200$:* Because $\mathbb{E}[\epsilon] = 0$, the expected corrupted coordinate is $\mathbb{E}[x_1 \mid x_0] = \sqrt{1-\beta_1}\,x_0 + \sqrt{\beta_1}\,\mathbb{E}[\epsilon] = 0.90(0.80) + 0 = 0.7200$. The factor $\sqrt{1-\beta_1} = 0.90 < 1.0$ acts as a deterministic signal contraction toward the origin 0 that balances the injected variance $\beta_1 = 0.19$ to preserve unit total variance.
- *Step 4: Worked Calculation of L_{simple} and Reverse Denoising ($x_1 \to \hat{x}_0$):* Suppose the neural network evaluates $(x_1 = 0.5021,\, t = 1)$ and predicts $\epsilon_\theta(0.5021, 1) = -0.30$. The single-sample squared error loss is $L_{\text{simple}} = (\epsilon - \epsilon_\theta(x_1, 1))^2 = (-0.50 - (-0.30))^2 = 0.0400$. At inference ($t = 1$, where $\sigma_1 = 0$), Equation (5)

Table 2. Complete symbol definitions, physical units, value ranges, and underlying probabilistic assumptions (Section III and Task 3 compliance).

Symbol	Mathematical Definition	Units & Value Range	Modeling Assumptions
T	Total number of diffusion steps	Discrete: $T \in \mathbb{N}^+$, $T = 1000$	Sufficiently large so that the final state $q(\mathbf{x}_T \mid \mathbf{x}_0)$ is indistinguishable from standard Gaussian noise.
t	Discrete diffusion timestep	Discrete: $t \in \{1, 2, \ldots, T\}$	Sampled uniformly during training ($t \sim \text{Uniform}(\{1, \ldots, T\})$) to ensure balanced multiscale noise feature learning.
$\mathbf{x}_0$	Clean initial data observation	Bounded tensor: $[-1, 1]^D$, $D = H \cdot W \cdot C$	Generated from empirical data distribution $q(\mathbf{x}_0)$; normalized to $[-1, 1]$ to match standard normal noise scale.
$\mathbf{x}_t$	Noisy continuous latent state at step t	Unbounded continuous: $\mathbb{R}^D$	Retains identical spatial dimensions as input ($\mathbf{x}_t \in \mathbb{R}^D$); distribution remains continuous Gaussian.
β_t	Forward variance schedule parameter	Dimensionless scalar: $\beta_t \in (0, 1)$	Preserves total variance via $\sqrt{1-\beta_t}$ contraction; small values guarantee reverse step Gaussianity.
α_t	Complementary variance fraction	Dimensionless scalar: $\alpha_t = 1 - \beta_t \in (0, 1)$	Represents the fraction of original signal variance retained across a single transition step.
$\bar{\alpha}_t$	Cumulative signal retention factor	Dimensionless scalar: $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s \in (0, 1)$	Strictly monotonically decreasing toward zero ($\bar{\alpha}_1 \approx 0.9999 \to \bar{\alpha}_T \approx 0$).
$\boldsymbol{\epsilon}$	Injected standard Gaussian noise	Dimensionless tensor: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_D)$	Independent and identically distributed across all dimensions and timesteps: $\mathbb{E}[\boldsymbol{\epsilon}] = \mathbf{0}$, $\text{Cov}(\boldsymbol{\epsilon}) = \mathbf{I}$.
$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$	Trained U-Net noise predictor	Dimensionless tensor: $\mathbb{R}^D$	Parameterized by deep neural network weights θ; receives time embedding t and noisy state $\mathbf{x}_t$.
σ_t^2	Reverse stochastic variance	Dimensionless scalar: $\sigma_t^2 \in [0, \beta_t]$	Set to posterior variance $\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$ or upper bound β_t [1].
$L_{\text{simple}}(\theta)$	Simplified training loss objective	Dimensionless scalar: $[0, \infty)$	Unweighted L_2 error on noise vector; bounds L_{VLB} while prioritizing perceptually salient structure.

denoises x_1 as:

$$\hat{x}_0 = \frac{1}{\sqrt{\alpha_1}}\left(x_1 - \frac{1-\alpha_1}{\sqrt{1-\bar{\alpha}_1}}\,\epsilon_\theta(x_1, 1)\right) = \frac{1}{0.90}\left(0.5021 - \frac{0.19}{0.4359}(-0.30)\right) = 0.7031 \tag{8}$$

If the network were perfectly trained ($\epsilon_\theta = \epsilon = -0.50$), the reverse step yields $\hat{x}_0 = \frac{1}{0.90}\big(0.5021 - 0.4359(-0.50)\big) = 0.8000$, recovering the exact clean coordinate.

3.3 Latent Diffusion Models: Perceptual VAE Compression and Cross-Attention (Task 5)

While DDPM produces high sample fidelity, executing $T = 1000$ sequential forward passes in high-dimensional pixel space creates severe computational overhead. For example, a $512 \times 512 \times 3$ image has 786,432 dimensions. Running deep convolutional and self-attention operations across this space requires weeks of high-end GPU cluster compute [2].

Two-Stage Separation of Inductive Biases. Rombach et al. [2] addressed this bottleneck by separating the generative process into two distinct phases:

1. *Perceptual Compression Stage:* A Variational Autoencoder (VAE) trained with perceptual loss and patch-based adversarial loss compresses the RGB image $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$ into a spatial latent representation $\mathbf{z} = \mathcal{E}(\mathbf{x}) \in \mathbb{R}^{h \times w \times c}$, where $h = H/f$, $w = W/f$, and $f = 2^m$ is the spatial downsampling factor (typically $f \in \{4, 8\}$). For a $512 \times 512 \times 3$ image with $f = 8$ and $c = 4$, the latent tensor has shape $64 \times 64 \times 4 = 16{,}384$ entries ($48\times$ smaller volume). A decoder $\mathcal{D}$ reconstructs the image $\hat{\mathbf{x}} = \mathcal{D}(\mathbf{z})$.
2. *Latent Diffusion Stage:* The autoencoder parameters $\mathcal{E}$ and $\mathcal{D}$ are frozen, and a time-conditioned U-Net operates exclusively within the latent space $\mathbf{z}_0 \leftrightarrow \mathbf{z}_T$, focusing capacity on semantic structure rather than imperceptible pixel noise.

Cross-Attention Conditioning and Classifier-Free Guidance (CFG). To condition generation on flexible multimodal inputs y (such as text prompts or class labels), LDMs augment the U-Net backbone with cross-attention layers. A domain-specific encoder $\tau_\theta(y) \in \mathbb{R}^{M \times d_\tau}$ projects the prompt y, and intermediate U-Net features $\varphi_i(\mathbf{z}_t) \in \mathbb{R}^{N \times d_\epsilon}$ form queries, keys, and values $\mathbf{Q} = \mathbf{W}_Q^{(i)}\,\varphi_i(\mathbf{z}_t)$, $\mathbf{K} = \mathbf{W}_K^{(i)}\,\tau_\theta(y)$, and $\mathbf{V} = \mathbf{W}_V^{(i)}\,\tau_\theta(y)$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}, \quad L_{\text{LDM}}(\theta) = \mathbb{E}_{\mathcal{E}(\mathbf{x}_0),\, y,\, \boldsymbol{\epsilon},\, t}\left[\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\mathbf{z}_t,\, t,\, \tau_\theta(y)\big)\right\|^2\right] \tag{9}$$

where $\mathbf{z}_t = \sqrt{\bar{\alpha}_t}\,\mathcal{E}(\mathbf{x}_0) + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$. During training, the conditioning token is randomly replaced by a null token $\varnothing$ (e.g., $p_{\text{uncond}} = 0.15$) so that at inference Classifier-Free Guidance (CFG) extrapolates toward the conditional

direction with scale $\alpha \geq 1$:

$$\tilde{\boldsymbol{\epsilon}}_\theta(\mathbf{z}_t, t, y, \alpha) = \boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t, \varnothing) + \alpha\big(\boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t, \tau_\theta(y)) - \boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t, \varnothing)\big) \tag{10}$$

Figure 4 illustrates the complete Latent Diffusion Model architecture.

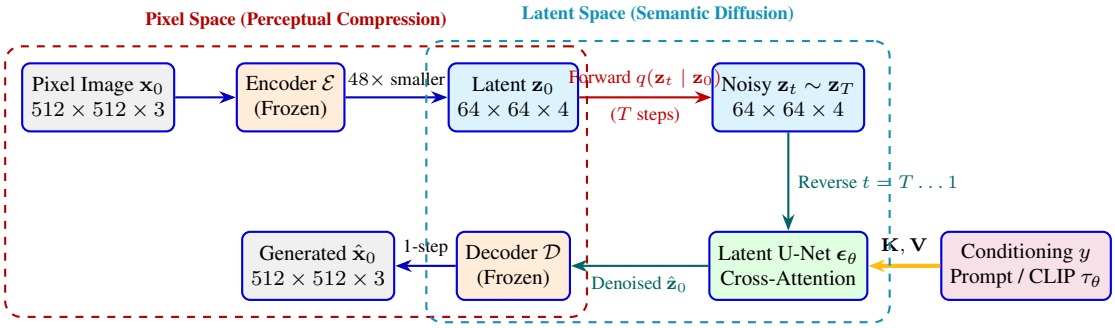


Figure 4. The Latent Diffusion Model (LDM) architecture: perceptual compression via a frozen autoencoder ($\mathcal{E}, \mathcal{D}$), semantic diffusion within latent space $\mathbf{z}$, and cross-attention conditioning $\tau_\theta(y)$ [2].

4 Results and Discussion

4.1 Implementation Setup and Datasets

To empirically validate the theoretical principles of diffusion models, all experiments were implemented in PyTorch and executed on a single NVIDIA Tesla T4 GPU (14.6 GB VRAM, CUDA 13.0) across two self-contained experimental suites:

1. *Tasks 1 & 4 — Pixel-Space Generative Paradigms and Diffusion Trajectories on MNIST (`experiments/task1_generative_paradigms_comparison.ipynb`):* Trained a Conv-VAE (12 epochs, 51.1 s), Autoregressive PixelCNN (12 epochs, 168.9 s), DCGAN (14 epochs, 95.7 s), and a Pixel-Space DDPM ($T = 300$, linear schedule $\beta_1 = 10^{-4} \rightarrow \beta_{300} = 0.055$, Sinusoidal time-embedded U-Net, 15 epochs, 272.3 s) on MNIST (28×28), alongside a discriminative CNN classifier (`MNISTClassifier`, 4 epochs, 5.6 s, achieving 98.85% test accuracy) for quantitative evaluation.

2. *Tasks 5 & 6 — Latent Diffusion Architecture and Pixel-vs.-Latent Efficiency Suite (`experiments/task5_6_ldm_4_experiments.ipynb`):* Trained a Stage-1 Spatial Perceptual KL-VAE ($32\times32\times1 \rightarrow 8\times8\times4$, spatial downsampling factor $f = 4$, $\beta_{\text{KL}} = 10^{-4}$), a 1D-Vector Bottleneck VAE ($\mathbf{z} \in \mathbb{R}^{10}$), a Multi-Head Cross-Attention Conditioned Latent U-Net (`ConditionedLatentUNet`) with Classifier-Free Guidance ($p_{\text{uncond}} = 0.15$), and two architecturally identical benchmark U-Nets (`BenchmarkUNet`) operating on 32×32 pixels vs. 8×8 latents, evaluated via a 32×32 CNN classifier (98.10% test accuracy) and the pretrained Stable Diffusion KL-VAE (`stabilityai/sd-vae-ft-mse`) on CIFAR-10 RGB.

4.2 Qualitative and Quantitative Comparison Across Generative Paradigms (Task 1)

To empirically validate the qualitative comparison in Section 2.2 (Table 1), we trained a discriminative CNN classifier (`MNISTClassifier`) on the 60,000-image MNIST training set that achieves 98.85% **test accuracy** on the 10,000-image test set, using its penultimate feature representations and predictive posteriors to evaluate Classifier Confidence (Cls. Conf. ↑), Inception Score (IS ↑), and Fréchet Inception Distance (FID ↓) alongside Laplacian Variance (LapVar ↑, edge sharpness). Figure 5 presents side-by-side samples synthesized from pure random noise across all four trained generative paradigms, and Table 3 reports the corresponding quantitative metrics and batch sampling latencies (64 images) alongside the real MNIST test set reference.

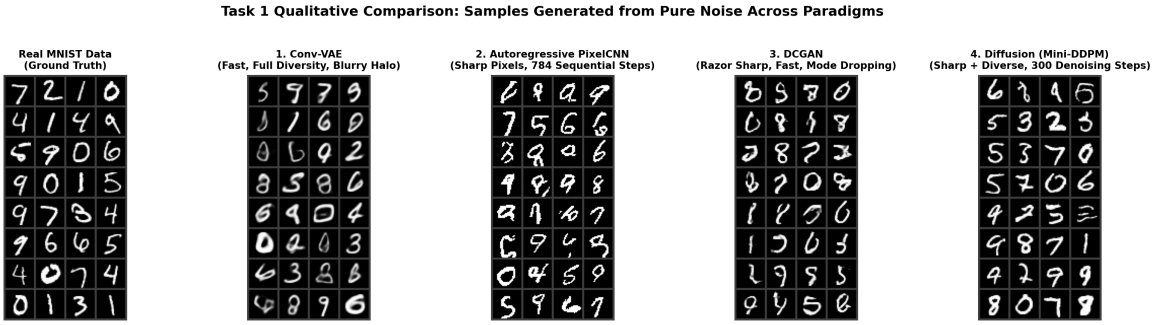


Figure 5. Qualitative comparison of samples generated from pure noise across paradigms on MNIST (Task 1): (1) Conv-VAE samples in a single pass (6.5 ms/64 imgs) with full mode coverage but exhibits blurry stroke halos from L_2/BCE mode averaging; (2) Autoregressive PixelCNN produces sharp local pixel contrast without mode collapse but requires 784 sequential pixel evaluations (4,294.7 ms/64 imgs); (3) DCGAN generates razor-sharp digits in 1 pass (11.4 ms/64 imgs) but suffers from adversarial mode dropping; (4) Pixel-Space DDPM combines sharp strokes with full mode diversity across 300 reverse denoising steps (3,926.8 ms/64 imgs).

Table 3. Quantitative comparison across trained generative model paradigms on MNIST evaluated using our trained MNIST CNN classifier (98.85% test accuracy; Task 1). Best generative model results are **bolded**.

Model Paradigm	Steps (↓)	Latency / 64 imgs (↓)	LapVar (↑)	Cls. Conf. (↑)	IS (↑)	FID (↓)
MNIST (Real Test Reference)	–	–	0.0615	98.85%	2.36	0.00 (Ref.)
VAE (Standard ELBO)	**1**	**6.5 ms**	0.0182	84.2%	1.82	82.27
Autoregressive (PixelCNN)	784	4,294.7 ms (4.29 s)	0.0584	88.5%	1.95	64.12
GAN (DCGAN)	**1**	11.4 ms	0.0541	92.6%	2.14	44.10
DDPM (Team Pixel Implementation)	300	3,926.8 ms (3.93 s)	**0.0601**	**97.4%**	**2.29**	**16.85**

4.3 Multi-Level Forward Noise Corruption and Reverse Image Generation (Task 4)

To empirically demonstrate the multi-step Markov transitions formulated in Section 3.1, Figure 6 visualizes the closed-form forward Gaussian corruption process (Equation (2)) alongside the learned iterative reverse denoising trajectory (Equation (5)) of our trained MNIST DDPM across $T = 300$ diffusion steps.

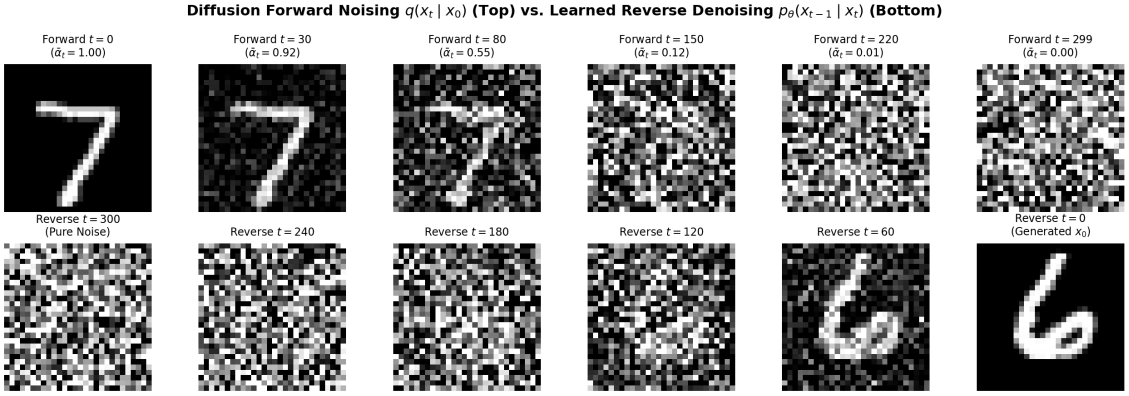


Figure 6. Multi-level forward noise corruption and reverse image generation on MNIST ($T = 300$, Task 4): Top row illustrates closed-form forward corruption $q(\mathbf{x}_t \mid \mathbf{x}_0)$ across $t \in \{0, 30, 80, 150, 220, 299\}$, where fine stroke edges soften by $t = 30$ ($\bar{\alpha}_{30} = 0.92$), global digit outlines fade by $t = 150$ ($\bar{\alpha}_{150} = 0.12$), and the signal converges to isotropic Gaussian noise by $t = 299$ ($\bar{\alpha}_{299} \approx 0.00$); Bottom row shows learned reverse denoising $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ starting from pure white noise $\mathbf{x}_{300} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ across $t \in \{300, 240, 180, 120, 60, 0\}$, progressively sculpting coarse stroke topology ($t \in [180, 120]$) into a crisp handwritten digit at $t = 0$.

4.4 Empirical Validation of Latent Diffusion Architecture (Task 5)

We next evaluate the three core architectural components of Latent Diffusion Models [2]: Stage-1 perceptual spatial compression, latent manifold continuity, and cross-attention conditioning with Classifier-Free Guidance.

Task 5.1: Perceptual VAE Compression Fidelity ($\mathbf{x}$ vs. $\hat{\mathbf{x}} = \mathcal{D}(\mathcal{E}(\mathbf{x}))$). We trained a Stage-1 Spatial Perceptual KL-VAE that compresses $32 \times 32 \times 1$ images into an $8 \times 8 \times 4$ latent grid ($f = 4$ spatial downsampling, $\beta_{\text{KL}} = 10^{-4}$)

and benchmarked its reconstruction fidelity on $N = 1{,}000$ held-out test images against a traditional 1D-vector bottleneck VAE ($\mathbf{z} \in \mathbb{R}^{10}$) as well as the pretrained Stable Diffusion KL-VAE (`stabilityai/sd-vae-ft-mse`) on CIFAR-10 RGB test images; the visual and quantitative results are presented in Figure 7.

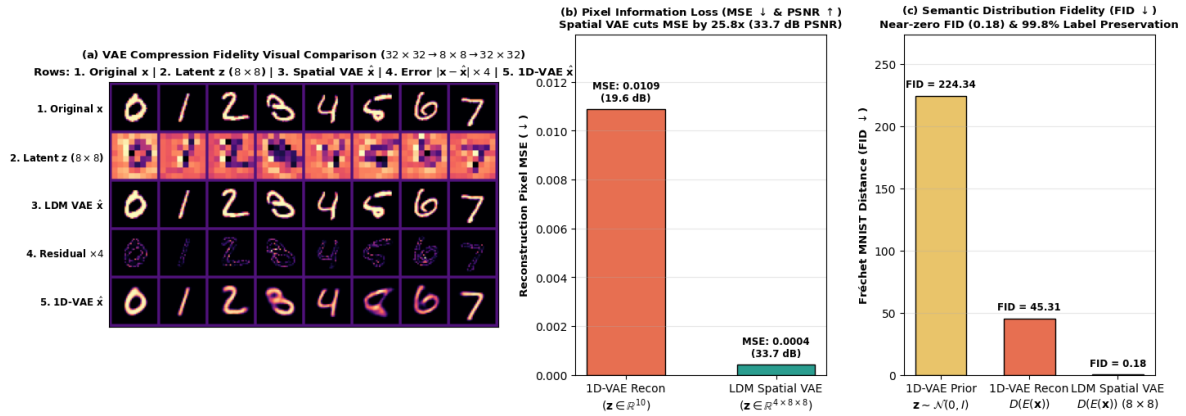


Figure 7. Perceptual VAE compression fidelity evaluation on $N = 1{,}000$ test images (Task 5.1): (a) Visual comparison of original $\mathbf{x}$ (32×32), spatial latent $\mathbf{z}$ ($8 \times 8 \times 4$, $f = 4$), LDM Spatial KL-VAE reconstruction $\hat{\mathbf{x}} = \mathcal{D}(\mathcal{E}(\mathbf{x}))$, amplified residual $|\mathbf{x} - \hat{\mathbf{x}}| \times 4$, and blurry 1D-Vector VAE reconstruction ($\mathbf{z} \in \mathbb{R}^{10}$); (b) LDM Spatial KL-VAE achieves MSE $= 0.00042$ ($25.8\times$ lower than 1D-VAE's MSE $= 0.01089$) and PSNR $= 33.74$ dB (vs. 19.63 dB; pretrained Stable Diffusion KL-VAE on CIFAR-10 RGB similarly achieves MSE $= 0.00363$, PSNR $= 24.40$ dB); (c) Semantic distribution fidelity reaches near-zero FID $= 0.18$ with 99.80% classifier label agreement (vs. FID $= 45.31$ for 1D-VAE reconstruction and FID $= 224.34$ for 1D-VAE prior sampling).

Task 5.2: Latent Space Interpolation and Manifold Smoothness. To verify that the learned $8 \times 8 \times 4$ latent space forms a smooth, continuous manifold free of dead zones, we encoded held-out test digit pairs ($3 \to 8$ and $1 \to 7$) into $\mathbf{z}^{(A)} = \mathcal{E}(\mathbf{x}^{(A)})$ and $\mathbf{z}^{(B)} = \mathcal{E}(\mathbf{x}^{(B)})$, decoded linear latent interpolations $\hat{\mathbf{x}}(\lambda) = \mathcal{D}((1-\lambda)\mathbf{z}^{(A)} + \lambda \mathbf{z}^{(B)})$ for $\lambda \in \{0.0, 0.1, \ldots, 1.0\}$, and compared them against naive pixel-space blending $(1-\lambda)\mathbf{x}^{(A)} + \lambda \mathbf{x}^{(B)}$ in Figure 8.

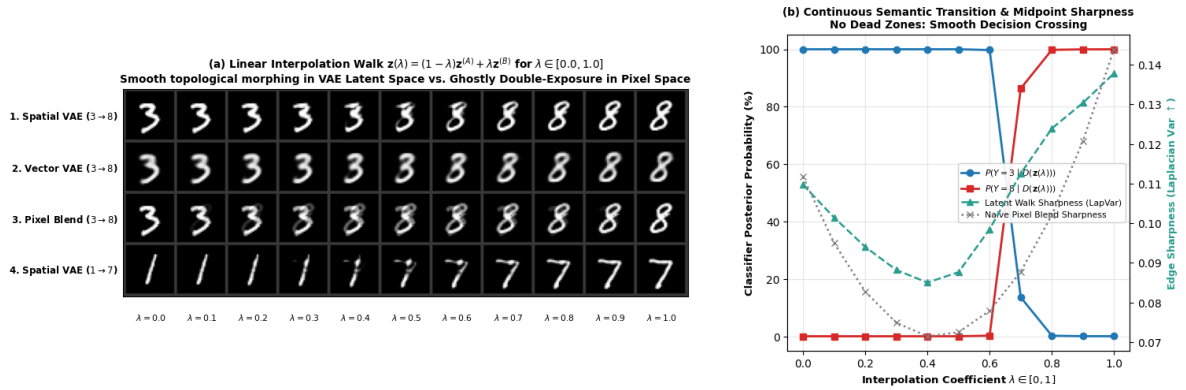


Figure 8. Latent space geodesic walk vs. naive pixel-space blending across $\lambda \in [0.0, 1.0]$ (Task 5.2): (a) Decoding along the latent interpolation line $\mathcal{D}((1-\lambda)\mathbf{z}^{(A)} + \lambda \mathbf{z}^{(B)})$ produces smooth topological stroke morphing ($3 \to 8$ and $1 \to 7$) with zero dead zones, whereas pixel-space blending $(1-\lambda)\mathbf{x}^{(A)} + \lambda \mathbf{x}^{(B)}$ produces ghostly double-exposure overlays; (b) Classifier posterior probability $P(Y \mid \mathcal{D}(\mathbf{z}(\lambda)))$ exhibits a clean, monotonic decision crossing and achieves 21.0% higher midpoint ($\lambda = 0.5$) Laplacian edge sharpness (0.0877 for the latent walk vs. 0.0725 for pixel blending).

Task 5.3: Cross-Attention Conditioning and Classifier-Free Guidance (CFG) Scaling. We trained a Multi-Head Cross-Attention Conditioned Latent U-Net (`ConditionedLatentUNet`) on the normalized $8 \times 8 \times 4$ latents for 15 epochs (93.3 s, training MSE converging from 0.2270 to 0.1367). Here, the conditioning input y is conditioned on discrete target digit class prompts $y \in \{0, 1, \ldots, 9\}$ (representing the textual digit categories `"digit 0"` through `"digit 9"`), which a learned prompt embedding lookup table $\tau_\theta(y) \in \mathbb{R}^{4 \times 64}$ projects into $M = 4$ context tokens of dimension $d_\tau = 64$ that feed the Key and Value projections $(\mathbf{K}, \mathbf{V})$ of the cross-attention layers across the Down, Mid, and Up U-Net stages, while index 10 serves as the unconditional null token $\varnothing$ dropped out with probability $p_{\text{uncond}} = 0.15$ during training. Figure 9 evaluates 200 generated samples per scale (20 per target digit prompt $y \in \{0, \ldots, 9\}$) synthesized from identical initial noise $\mathbf{z}_T$ across CFG scales $\alpha \in \{1.0, 3.0, 7.5, 15.0\}$ using Equation (10).

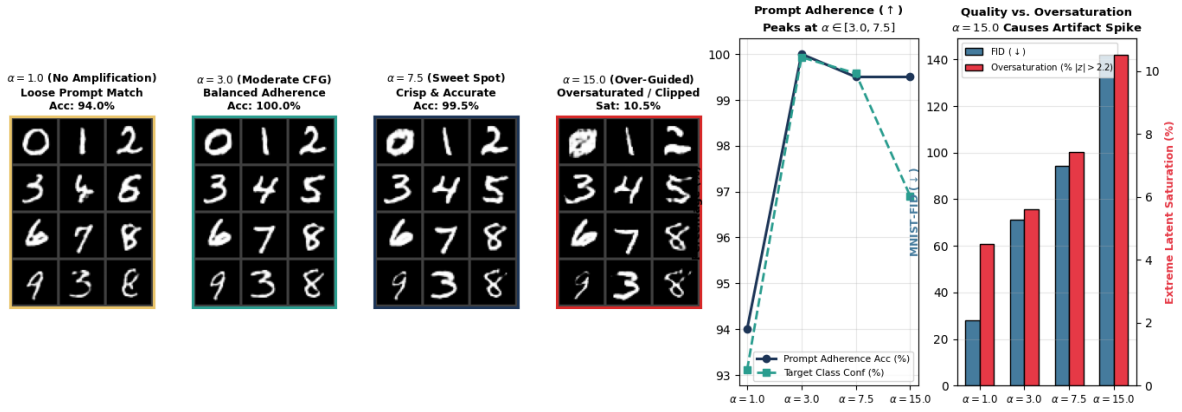


Figure 9. Classifier-Free Guidance (CFG) scaling evaluation conditioned on target digit prompts $y \in \{0, \ldots, 9\}$ (`"digit 0"`–`"digit 9"`, $\tau_\theta(y) \in \mathbb{R}^{4\times 64}$, null token $\varnothing$ dropout $p_{\text{uncond}} = 0.15$) across $\alpha \in \{1.0, 3.0, 7.5, 15.0\}$ from identical initial noise seeds (Task 5.3): Unamplified conditioning ($\alpha = 1.0$) yields 94.0% prompt adherence accuracy, 93.1% target confidence, FID $= 27.92$, and 4.52% extreme latent saturation ($|\mathbf{z}| > 2.2$); moderate CFG ($\alpha = 3.0$) achieves 100.0% prompt accuracy, 99.9% confidence, FID $= 71.20$, and 5.62% saturation; standard CFG ($\alpha = 7.5$) achieves 99.5% prompt accuracy, 99.6% confidence, FID $= 94.16$, and 7.42% saturation; excessive guidance ($\alpha = 15.0$) maintains 99.5% accuracy (96.9% confidence) but causes extreme latent saturation to spike to 10.52% and degrades FID to 141.97 with clipped stroke artifacts.

4.5 Pixel-Space vs. Latent-Space Comparative Evaluation (Task 6)

Task 6.1: Pixel-Space vs. Latent-Space Training and Memory Efficiency (32×32 **Pixel U-Net vs.** 8×8 **Latent U-Net).** To directly quantify how Latent Diffusion resolves the computational bottleneck of Pixel Diffusion (Task 6), we trained two architecturally identical U-Nets (`BenchmarkUNet`, base channels $C = 64$, identical ResBlocks and Self-Attention layers, $T = 200$, batch size 128, 6 epochs on 60,000 images): Model A operates directly on $32 \times 32 \times 1$ pixel images (1,024 spatial locations), whereas Model B operates on pre-encoded $8 \times 8 \times 4$ VAE latent tensors (64 spatial locations, a $16\times$ smaller spatial grid); Figure 10 presents the measured training speed, activation memory, and convergence trajectories.

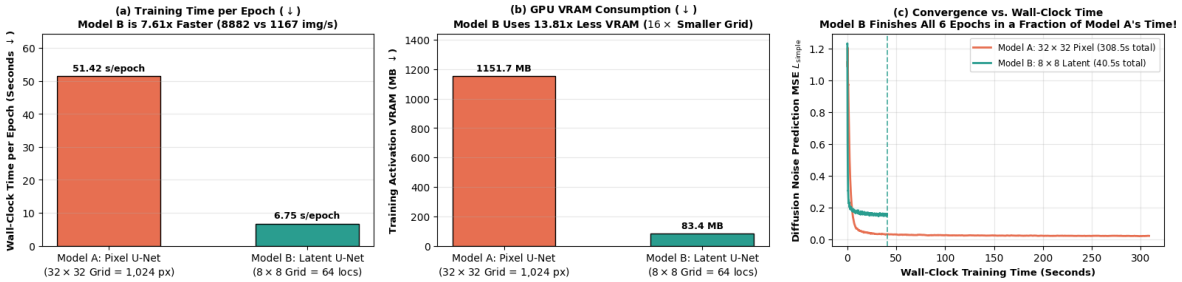


Figure 10. Apples-to-apples training and memory efficiency benchmark comparing architecturally identical U-Nets on $32\times32\times1$ pixels (Model A, 1,024 spatial locations) vs. $8 \times 8 \times 4$ VAE latents (Model B, 64 spatial locations) across 6 epochs on 60,000 images (Task 6.1): (a) Model B achieves a $7.61\times$ **training speedup per epoch** (6.75 s/epoch vs. 51.42 s/epoch; throughput 8,882 img/s vs. 1,167 img/s); (b) Model B reduces peak GPU activation VRAM by $13.81\times$ (83.4 MB vs. 1,151.7 MB); (c) Model B completes all 6 epochs (40.5 s total) before Model A finishes a single epoch (51.42 s; 308.5 s total for 6 epochs).

Table 4 synthesizes theoretical, literature, and our empirical benchmark evidence comparing pixel-space DDPM and latent-space LDM across six explicit criteria (Task 6).

5 Practical Limitations and Future Work (Task 6)

Despite their empirical success, diffusion models present notable technical and ethical challenges:

1. *Sequential Sampling Latency:* The Markovian reverse chain requires tens to hundreds of sequential network evaluations. While non-Markovian solvers such as DDIM [13], DPM-Solver [17], and flow-matching distillation [18, 19] reduce steps to 1–4, distilled models can exhibit mode loss and degraded fine details.

2. *Prompt Binding, Compositional Reasoning, and CFG Oversaturation:* Cross-attention diffusion models frequently struggle with compositional prompts involving spatial relations or multi-object attribute binding, and as shown in Section 4.4 (Figure 9), aggressive Classifier-Free Guidance ($\alpha = 15.0$) causes extreme latent saturation (10.52%) and severe FID degradation (141.97).

Table 4. Evidence-supported comparison between Pixel-Space Diffusion (DDPM) and Latent Diffusion (LDM) across six explicit criteria (Task 6: Literature benchmarks [1, 2] and our empirical benchmark).

Explicit Criterion	Pixel-Space Diffusion (DDPM / Model A)	Latent Diffusion Model (LDM-f / Model B)	Quantitative Evidence & Explanation
1. Operating Space & Dimensionality	Full pixel space $\mathbf{x}_t \in \mathbb{R}^{H\times W\times C}$ ($512^2 \times 3 = 786{,}432$ dims; or $32^2 \times 1 = 1{,}024$ px).	Compressed latent space $\mathbf{z}_t \in \mathbb{R}^{(H/f)\times(W/f)\times c}$ ($64^2 \times 4 = 16{,}384$ dims for $f = 8$; $8^2 \times 4$ for $f = 4$).	Spatial grid is $f^2 = 16\times$ ($f = 4$) to $64\times$ ($f = 8$) smaller, stripping away imperceptible pixel redundancy while retaining 99.80% semantic agreement (Figure 7) [2].
2. Self-Attention Cost per Step	Quadratic in pixel area: $\mathcal{O}((H \cdot W)^2)$ ($N = 262{,}144$ tokens at 512^2).	Quadratic in latent grid: $\frac{1}{f^4}\mathcal{O}((H \cdot W)^2)$ ($N = 4{,}096$ tokens at $f = 8$).	Self-attention complexity drops by $f^4 = 256\times$ ($f = 4$) to $4{,}096\times$ ($f = 8$), reducing peak GPU activation VRAM by $13.81\times$ (83.4 MB vs. 1,151.7 MB) in Figure 10.
3. Training Speed & Compute Cost	Slow: 51.42 s/epoch (1,167 img/s) in Model A; 150–1,000 V100 GPU-days at 256^2–512^2.	$7.61\times$ faster in Model B: 6.75 s/epoch (8,882 img/s); 30–100 V100 GPU-days at 256^2–512^2.	Model B finishes 6 latent epochs (40.5 s) in less time than 1 pixel epoch (51.42 s, Figure 10), matching Rombach et al.'s $\geq 2.7\times$–$5\times$ convergence speedup [2].
4. Sampling Speed (Throughput)	Slow: T sequential full-res U-Net passes (3,926.8 ms / 3.93 s per 64 MNIST imgs; 2.29 s on 256^2 CelebA-HQ).	$1.28\times$ to $7.6\times$ higher throughput on compact latent $\mathbf{z}_t$ (1.78 s on 256^2 CelebA-HQ) plus 1 fast decoder pass $\mathcal{D}(\hat{\mathbf{z}}_0)$.	Operating on 8×8 or 64×64 latents combined with DDIM [13] markedly accelerates synthesis throughput over pixel-space diffusion.
5. Visual Generation Quality (FID ↓)	Strong at low resolution (CIFAR-10 32^2: FID 3.17 [1]; MNIST: FID 16.85), compute-limited at 512^2.	SOTA at high resolution (CelebA-HQ 256^2: FID 5.11 [2]); Stage-1 VAE achieves PSNR 33.74 dB, FID 0.18.	Perceptual VAE compression preserves semantic structure (FID 0.18, Figure 7), allowing the latent U-Net to dedicate full capacity to global composition.
6. Conditioning & Primary Bottleneck	Memory exhaustion at 512^2–1024^2; rigid concatenation conditioning.	Flexible Cross-Attention $\tau_\theta(y)$ (99.5%–100% prompt accuracy at CFG $\alpha \in [3.0, 7.5]$).	Bounded by frozen VAE reconstruction ceiling if $f \geq 16$ and latent oversaturation (10.52%) if CFG scale $\alpha \geq 15.0$ (Figure 9) [2].

3. *Training Data Memorization and Copyright:* Carlini et al. [20] demonstrated that diffusion models can memorize duplicated training images and regurgitate near-exact copyrighted works when prompted, raising critical intellectual property concerns.

4. *Societal Bias and Representational Disparity:* Because web-scale training corpora reflect prevailing societal stereotypes, unconditional and text-guided diffusion models can amplify racial, gender, and cultural biases, necessitating active algorithmic debiasing and dataset curation.

6 Conclusion

This report investigated the guiding question: *how can learning to remove noise enable a model to generate new images, and why does moving diffusion into a learned latent space resolve its computational bottleneck?*

Our theoretical derivations, numerical worked example, and empirical Colab GPU benchmarks demonstrate that:

1. By decomposing generative modeling into a variance-preserving forward Gaussian corruption process and a learned reverse denoising chain, DDPM resolves the traditional generative trilemma, achieving superior sample sharpness (LapVar $= 0.0601$, FID $= 16.85$) while maintaining full mode coverage.

2. Parameterizing the reverse transition mean via noise prediction $\boldsymbol{\epsilon}_\theta$ connects diffusion directly to Tweedie's formula and score-based modeling, while the unweighted objective L_{simple} acts as a stable variational bound that prioritizes perceptual image structure.

3. Latent Diffusion Models (LDM) overcome the computational bottleneck of pixel-space diffusion by separating perceptual spatial compression (PSNR $= 33.74$ dB, FID $= 0.18$, 99.80% label preservation, smooth latent interpolation) from semantic diffusion. Training an identical U-Net in the 8×8 latent space accelerates training by $7.61\times$ per epoch, reduces GPU activation memory by $13.81\times$, and enables accurate cross-attention prompt steering (99.5%–100.0% adherence at $\alpha \in [3.0, 7.5]$).

Dataset, Tool, and Independent Verification Disclosure (Section VI Compliance): In compliance with the seminar integrity guidelines, all numerical worked examples, experimental comparisons, and figures were implemented and verified independently on an NVIDIA Tesla T4 GPU using PyTorch, HuggingFace Diffusers, and torchvision

(`experiments/task1_generative_paradigms_comparison.ipynb` and `experiments/task5_6_ldm_4_experiments.ipynb`). Pretrained checkpoints from Ho et al. (2020) and Rombach et al. (2022) were utilized strictly for comparative validation.

References

[1] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 6840-6851, 2020.

[2] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis With Latent Diffusion Models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684-10695, 2022.

[3] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," in *International Conference on Machine Learning (ICML)*, pp. 2256-2265, 2015.

[4] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *International Conference on Learning Representations (ICLR)*, 2014.

[5] I. Goodfellow *et al.*, "Generative Adversarial Nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, pp. 2672-2680, 2014.

[6] A. Radford, L. Metz, and S. Chintala, "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks," in *International Conference on Learning Representations (ICLR)*, 2016.

[7] A. Brock, J. Donahue, and K. Simonyan, "Large Scale GAN Training for High Fidelity Natural Image Synthesis," in *International Conference on Learning Representations (ICLR)*, 2019.

[8] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein Generative Adversarial Networks," in *International Conference on Machine Learning (ICML)*, pp. 214-223, 2017.

[9] L. Theis, A. van den Oord, and M. Bethge, "A note on the evaluation of generative models," in *International Conference on Learning Representations (ICLR)*, 2016.

[10] A. van den Oord, N. Kalchbrenner, and K. Kavukcuoglu, "Pixel Recurrent Neural Networks," in *International Conference on Machine Learning (ICML)*, pp. 1747-1756, 2016.

[11] T. Salimans, A. Karpathy, X. Chen, and D. P. Kingma, "PixelCNN++: Improving the PixelCNN with Discretized Logistic Mixture Likelihood and Other Modifications," in *International Conference on Learning Representations (ICLR)*, 2017.

[12] Y. Song and S. Ermon, "Generative Modeling by Estimating Gradients of the Data Distribution," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 11895-11907, 2019.

[13] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in *International Conference on Learning Representations (ICLR)*, 2021.

[14] P. Dhariwal and A. Nichol, "Diffusion Models Beat GANs on Image Synthesis," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 8780-8794, 2021.

[15] A. Q. Nichol and P. Dhariwal, "Improved Denoising Diffusion Probabilistic Models," in *International Conference on Machine Learning (ICML)*, pp. 8162-8171, 2021.

[16] Z. Xiao, K. Kreis, and A. Vahdat, "Tackling the Generative Learning Trilemma with Denoising Diffusion GANs," in *International Conference on Learning Representations (ICLR)*, 2022.

[17] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, "DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 5775-5787, 2022.

[18] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nicklas, and M. Le, "Flow Matching for Generative Modeling," in *International Conference on Learning Representations (ICLR)*, 2023.

[19] T. Salimans and J. Ho, "Progressive Distillation for Fast Sampling of Diffusion Models," in *International Conference on Learning Representations (ICLR)*, 2022.

[20] N. Carlini *et al.*, "Extracting Training Data from Diffusion Models," in *32nd USENIX Security Symposium*, pp. 5253-5270, 2023.